

1.1.3 Les bases d'un environnement de test

EN BREF

Un **environnement de test** n'est pas qu'une URL : c'est un **contexte complet** (configuration, données, objectif) dans lequel on exécute des vérifications **sans risquer la production**. Sur Hublot, nous distinguons **cinq niveaux** — DEV, TEST (CI), TEST local, STAGING et PROD — chacun avec un rôle précis dans le cycle de livraison.

Objectif pédagogique

Comprendre comment **séparer et configurer** les environnements de développement, de test, de préproduction et de production, et savoir **quel type de test** exécuter dans chacun.

Contexte — projet Hublot

Hublot manipule des **données RH** (effectifs, statuts, missions). Tester directement en production signifierait exposer des erreurs à des utilisateurs habilités ou corrompre des données de référence. Nous avons donc formalisé une **progression** : coder en local (DEV), valider automatiquement (CI), reproduire le build prod en local (TEST local), faire valider par le métier (STAGING), puis livrer (PROD).

Définitions

Terme	Définition
Environnement	Ensemble configuration + infrastructure + données pour un usage donné (dev, test, prod).
DEV	Poste du développeur ; itérations rapides, identifiants de démo.
TEST (CI)	Environnement éphémère GitHub Actions qui exécute la même chaîne pour tous.
TEST local	Reproduction du build de production sur la machine (preview ou Docker).
STAGING	Préproduction : même stack que PROD, URL et comptes distincts, validation métier.
PROD	Environnement utilisé par les agents DIRM en conditions réelles.

Ce qui a été mis en place

Vue d'ensemble

CODE

```
DEV (localhost:3000)
  → push Git
TEST / CI (GitHub Actions – VM éphémère)
  → merge staging
STAGING (Netlify, branche staging)
  → merge main + CI verte
PROD (Netlify, branche main – dirmhublot.netlify.app)
```

Les cinq environnements

Env.	Rôle	Où	URL / accès	Configuration
DEV	Coder, déboguer	Poste local	http://localhost:3000	<code>.env.development</code>
TEST	Qualité automatisée	GitHub Actions — workflow CI	Actions du dépôt	<code>.github/workflows/ci.yml</code>
TEST local	QA « comme la prod »	Docker ou preview Vite	http://localhost:4173	<code>.env.test</code> , <code>docker-compose.test.yml</code>
STAGING	Préproduction métier	Netlify — branche <code>staging</code>	URL Netlify staging	<code>netlify.toml</code> + variables Netlify
PROD	Utilisateurs habités	Netlify — branche <code>main</code>	https://dirmhulot.netlify.app	Variables Netlify (secrets)

Éléments visibles dans l'application

Le fichier `src/config/environment.ts` lit `VITE_APP_ENV` et affiche un **badge** en DEV et STAGING (« Développement », « Préproduction »), **absent en PROD** pour ne pas perturber les utilisateurs.

Pourquoi cette séparation

Risque sans séparation	Ce que la séparation apporte
Tester en PROD	Régression visible par les agents DIRM
Même mot de passe partout	Fuite d'identifiants staging → compromission prod
CI différente selon les postes	« Ça marche chez moi » — la CI impose la même chaîne pour tous
Déployer sans build de prod testé	TEST local (port 4173) sert exactement les fichiers statiques du build

Comment ça fonctionne

DEV — développement (poste local)

Objectif : itérer rapidement sans impact utilisateur.

TERMINAL

```
git clone https://github.com/Meriem1403/Hublot.git
cd Hublot
npm ci
npm run dev # http://localhost:3000
```

Variables dans `.env.development` (versionné, comptes **démo** uniquement) :

ENV

```
VITE_APP_ENV=development
VITE_APP_USERNAME=admin
VITE_APP_PASSWORD=demo
```

Avant chaque push : `npm run lint` et `npm run test:run`.

TEST — intégration continue (GitHub Actions)

Objectif : gate automatique — impossible de merger du code qui échoue lint, tests, audit, build ou E2E.

Déclencheurs : push / PR sur `main` et `staging`.

Pipeline (`.github/workflows/ci.yml`) :

Étape	Commande
ES - Lint	<code>npm run lint</code>
Tests unitaires	<code>npm run test:run</code> (48 tests)
Audit npm	<code>npm run audit:prod</code>
Build	<code>npm run build</code>
E2E	<code>npm run test:e2e</code>

L'environnement TEST est **éphémère** : une VM Ubuntu est créée, exécute la chaîne, puis disparaît. Pas de base partagée — les tests unitaires utilisent des jeux de données contrôlés.

Netlify PROD est configuré en **Deploy only when required checks pass** : le check **CI** doit être vert.

TEST local — complément manuel

Objectif : vérifier le **build de production** sans déployer sur Netlify.

Option A — Preview Vite :

TERMINAL

```
npm run build:test
npm run preview:test
# http://localhost:4173 — identifiants e2e-user / e2e-pass (.env.test)
```

Option B — Docker (proche du NAS) :

TERMINAL

```
npm run test:preview:docker
```

Checklist machine complète :

TERMINAL

```
npm run check:env
```

Vérifie `.env`, `npm ci`, lint, tests, audit et `build:test`.

STAGING — préproduction

Branche Git `staging` → déploiement Netlify séparé de `main`. Variable `VITE_APP_ENV=staging`, badge « Préproduction ». Détail de la configuration : document **1.2.3**.

Promotion : merge `staging` → `main` après validation métier et CI verte.

PROD — production

Branche `main`, secrets **uniquement** dans Netlify (jamais dans Git), pas de badge environnement.

Quand utiliser quel environnement

Type de vérification	DEV	TEST (CI)	TEST local	STAGING	PROD
Unitaires Vitest	Oui	Auto	via <code>check:env</code>	—	—
ESLint	Oui	Auto	via <code>check:env</code>	—	—
E2E Playwright	Oui	Auto	preview :4173	Manuel	—
Build production	Oui	Auto	<code>build:test</code>	Auto Netlify	Auto Netlify
Audit npm	Oui	Auto	via <code>check:env</code>	—	—
Tests manuels UI	Navigateur	—	:4173	URL staging	Lecture seule
Headers sécurité (curl)	—	—	—	curl staging	curl prod

Preuves et démonstration (parcours jury, ~5 min)

- 1 **DEV** — `npm run dev` → badge « Développement » + login démo.
- 2 **TEST** — GitHub Actions → workflow **CI** → tous les jobs verts.
- 3 **TEST local** — `npm run check:env` ou preview sur :4173.
- 4 **STAGING** — branche `staging` + URL Netlify (si branch deploy activé).
- 5 **PROD** — <https://dirmhublot.netlify.app> — pas de badge, comptes réels.

Bonnes pratiques

1. Développer sur **DEV**, lancer `npm run check:env` avant push.
2. Pousser sur `staging` pour une démo métier ; merger vers `main` seulement si CI verte.
3. Ne jamais committer `.env` avec mots de passe réels (voir `.gitignore`).
4. Conserver des identifiants **différents** entre STAGING et PROD.

SYNTHÈSE

« Hublot distingue **cinq environnements** avec des rôles clairs : DEV pour coder, **CI** pour la qualité automatique identique pour tous, TEST local pour le build statique, STAGING pour la validation métier, PROD pour les agents DIRM. Chaque niveau a sa **configuration** (`VITE_APP_ENV` , variables Netlify) et on ne progresse vers la prod qu'avec une **CI verte**. »

Documents liés

- › 1.2.3 Mettre en place un environnement de test — staging Netlify
- › 1.1.3.1 Guide Docker — TEST local via conteneurs
- › 1.2.8 Procédure d'exécution des tests (épreuve).md) — démo oral
- › Annexe 06 — Environnement test